Técnicas de Busca e Ordenação 25/2 - Trabalho Prático 1

Caminho Mais Curto de Dijkstra

Relatório

Francisco Vassoler Merizio - 2024102652 e Rodrigo Mattioli - 2019202335

1_ Primeira Implementação

1.1 O básico

Passo a passo dos comandos para a execução do trabalho:

```
     tar -xzvf T1-2024102652-2019202335-TBO-2025-2.tar.gz
//Se quiser a primeira implementação
     make vanilla
     ./trab1 <nome_arquivo_entrada> <nome_arquivo_saida>
//Se quiser a segunda implementação
     make basicvec
     ./trab1 <nome_arquivo_entrada> <nome_arquivo_saida>
//Se quiser a implementação com melhoria
     make improve
     ./trab1 <nome_arquivo_entrada> <nome_arquivo_saida>
```

A ideia inicial do trabalho era simples: Implementar o Algoritmo de Dijkstra.

Algoritmo extremamente conhecido que tem como objetivo, em um grafo G qualquer, encontrar o caminho mais curto de um vértice até todos os vértices do grafo.

Muito utilizado nos dias atuais em Sistemas de Navegação(GPS), Roteamento de Redes, ou, como no exemplo do trabalho, para produtos em um Market Place.

A complexidade do algoritmo, quando usada uma heap binária é de

$$O((M+N)\log N)$$

onde M é o número de arestas e N o número de Vértices, e isso será explicado, e será importante para o futuro da análise.

A implementação da primeira versão do trabalho poderia ser dividido em três etapas:

- Leitura dos Dados de Entrada;
- Execução do Algoritmo de Dijkstra;
- Saída dos resultados.

1.2 Leitura de Dados, “o Primeiro Gargalo”

Quando iniciamos já nos deparamos com um problema: A leitura dos dados é extremamente lenta e custosa, e isso se deve ao padrão do arquivo de entrada:

```
node_0
node_0, 10, 0, 5, 0
node_1, 0, 1, 2, 0
node_2, 0, 0, 0, 4
node_3, 0, 3, 3, 0
node_4, 7, 0, 6, 0
<Nome_Vertice>, <ligação vértice 0>,...
```

O problema é claro: Como em todos os nós é apresentado todas as ligações, existindo ou não, para com os outros vértices, a leitura da entrada dos dados é sempre uma **matriz de adjacência**, então a leitura sempre será quadrática**.**

Não ter previamente a quantidade de nós totais poderia ser um problema, onde a alocação de memória poderia ser imprecisa, gastando mais memória que o necessário, ou fazendo vários *reallocs*, uma operação custosa.

Para isso não ser um problema, é feita uma leitura inicial apenas da primeira linha e faz-se uma contagem da quantidade de vértices no total, "rebobinando" o arquivo ao fim da contagem.

Isso garante o tamanho exato da estrutura de dados que vai precisar ser alocada, sem desperdício e nem necessidade de *reallocs* no caminho.

```
//TESTE PARA ENCONTRAR O NUMERO DE NOS
fscanf(arq,"%[^\n] ", s);
fscanf(arq,"%[^\n~,] ", node_aux);
while(1){
    while(fscanf(arq,", %f", &opt)==1){
        cont++;
    }
    //CONTROLE PARA BOMBA
    fscanf(arq,"%[^,]", bomba);
    if(strcmp(bomba, "bomba")!=0) break;
    cont++;
}
rewind(arq);
```

1.3 Execução do Algoritmo de Dijkstra

Criada a Heap Binária e leitura dos vértices e suas conexões concluída, para entender o porquê do custo, precisamos do pseudocódigo:

```
Dijkstra(G, w, s)
    Inicializa(G, s)
    S ← ∅
    ConstruaFilaPrioridade(Q, G.V )
    while Q 6= ∅
        u ← ExtrairMinimo(Q)
        S ← S ∪ {u}
        for cada vértice v adjacentes à u
            Relaxe(u, v)
            DecresçaChave(Q, v, d[v])

Inicializa(G, s)
    for cada vértice v ∈ G.V
        v.pai ← Nil
        v.d ← ∞
        s.d ← 0
```

Como utilizamos a estrutura da Heap Binária, podemos fazer uma pequena análise de custo do algoritmo:

- Construir a Fila de Prioridade = N vezes a inserção, que tem custo log N (log crescente proporcionalmente a N, mas usaremos como base o custo final) = **N log N**;
- Extrair o Mínimo = N vezes a extração, que também é log N = **N log N**;
- Relaxamento = M vezes, e supondo que todas as vezes seja necessário relaxar em todas as vezes que a função é chamada, custo de atualizar a heap log N = **M log N;**

Somando todas as etapas temos:

$$\mathbf{N \log N + N \log N + M \log N = (2N + M) \operatorname{Log} N}$$

$$\mathbf{O((M+N)\operatorname{Log} N)}$$

Além disso, uma das observações envolvia um cálculo não determinístico, em que, em diferentes execuções, diferentes caminhos mínimos poderiam ser encontrados, se houvesse mais de uma possibilidade.

O que faz escolher um caminho diferente de mesmo custo ou não, é a avaliação para o relaxamento, de forma não padronizada, ora deve ser feita a comparação apenas com “<”, ora deve ser feita com “<=”, criando uma seleção não determinística, ao custo de uma comparação a mais por iteração.

```
float relaxeNo(No* n1, No* n2, float d_1to2){
    int n = rand();
    if(n%2==0){
        if((retornaDistanciaS(n1)+d_1to2)<retornaDistanciaS(n2)){
            atualizaPai(n2, n1);
            return retornaDistanciaS(n1)+d_1to2;
        }
    }else{
        if((retornaDistanciaS(n1)+d_1to2)<=retornaDistanciaS(n2)){
            atualizaPai(n2, n1);
            return retornaDistanciaS(n1)+d_1to2;
        }
    }

    return 0;
}
```

1.4 Saída e a Padronização

A saída acaba por ser a parte mais simples da implementação, tendo que seguir a padronização estipulada na documentação:

```
SHORTEST PATH TO node_0: node_0 <- node_0 (Distance: 0.00)
SHORTEST PATH TO node_3: node_3 <- node_0 (Distance: 5.00)
SHORTEST PATH TO node_1: node_1 <- node_3 <- node_0 (Distance: 8.00)
SHORTEST PATH TO node_2: node_2 <- node_3 <- node_0 (Distance: 8.00)
SHORTEST PATH TO node_4: node_4 <- node_2 <- node_3 <- node_0 (Distance: 12.00)
```

Para a execução dessa parte, no struct “No” cada vértice tem uma referência para o seu pai, a partir daí é simples:

Enquanto o pai não for nulo (se tudo funcionar corretamente, apenas o *node_S* terá seu pai como NULL) continua imprimindo o nome do vértice pai, e ao final, imprime a distância final para S.

O custo da impressão depende da profundidade de cada caminho, a função é executada N vezes (o menor caminho de nodes para todos os vértices do grafo) e o loop interno é executado relativo a profundidade de cada caminho.

OBS: Como o *heapify* já vai retirando o menor nó a cada iteração, não existe a necessidade de, ao fim da execução, ordenar o array novamente, que ele já estará organizado relativo à distância para o *Node_s*.

Resultados de tempo para os casos de teste:

Dijkstra	Pequeno 1 (86 KB)	Pequeno 4 (8.593 KB)	Médio 1 (214 MB)	Médio 2 (214 MB)	Médio 3 (214 MB)	Médio 4 (309 MB)
Heap Binária	0.002175	0.177552	5.619822	5.228124	4.906362	9.218174

2_ Segunda Implementação

A segunda parte do trabalho envolvia fazer uma segunda implementação do algoritmo de Dijkstra, porém, utilizando outra estrutura de dados e outra implementação para a mesma execução.

2.1 Árvore Red-Black e os problemas

Em um primeiro momento, a ideia foi a utilização de uma árvore red-black, o conceito era simples: Retirar o elemento mínimo, rebalancear todos os nós que fossem necessários e repetir o processo até que a árvore estivesse vazia.

Porém, logo no início da tentativa, deparei com o problema que tornaria a implementação inviável: O conceito de “Key/Value” e a inexistência de chaves duplicadas.

Se a chave fosse a distância para o *node_S*, algo esperado, eu não poderia garantir que as chaves fossem únicas, inclusive, ao inicializar, todas as chaves seriam iguais (infinito).

A segunda opção seria algo que garantisse a unicidade de cada nó: o Id. Mas essa possibilidade também traria problemas claros, os vértices seriam ordenados pelo Id, e a retirada deveria ser do nó com menor distância, criando um conflito, perdendo toda a eficiência que se ganharia com o uso da árvore.

Opção descartada, vamos agora para a segunda solução proposta.

2.2 Array Sequencial não ordenado

Se olharmos de forma abstrata, a heap binária nada mais é que um array, a questão que a diferencia é a pré ordenação em que os elementos são inseridos na fila, a chamada *heap order*.

A segunda implementação partiria desse pensamento: E se usarmos um array simples, e garantirmos que nesse array tenha um método para garantir a ordenação dos elementos, poderíamos manter uma eficiência semelhante à original?

```
struct array{
        No** array_no;
        int n;
};
```

A estrutura é bem simples, a única mudança seria na forma de buscar o próximo elemento, que no pseudo-código é o “`u ← ExtrairMinimo(Q)`”

Diferença entre os Algoritmos em si:

```c
//DISTANCIA DO NODE_S SETADA EM 0
atualizaHeap(heap, node_s, 0.0);

while(tamanhoHeap(heap)>0){
    No* v_atual=removeHeap(heap);
    //imprimirNo(v_atual);
    Cel* v_aux=retornaCel(retornaWarden(v_atual));
    while(1){
        if(v_aux==NULL) break;
        No* aux = retornaConex(v_aux);

        float peso=relaxeNo(v_atual, aux, retornaDistancia(v_aux));
        if(peso)
            atualizaHeap(heap, aux, peso);
        v_aux=retornaProxCel(v_aux);
    }
}
```

```c
//DISTANCIA DO NODE_S SETADA EM 0
atualizaDistancia(node_s, 0);
trocaPosicaoArray(array, ns, 0);

for(int i=0;i<cont;i++){
    if(i>0) ordenarArray(array, i);
    No* v_atual=retornaNoArray(array, i);
    Cel* v_aux=retornaCel(retornaWarden(v_atual));
    while(1){
        if(v_aux==NULL) break;
        No* aux = retornaConex(v_aux);

        float peso=relaxeNo(v_atual, aux, retornaDistancia(v_aux));
        if(peso)
            atualizaDistancia(aux, peso);
        v_aux=retornaProxCel(v_aux);
    }
}
```

2.3 QuickSort ou “SelectionSort”

Sobre a função “*ordenarArray()”,* em um primeiro momento a ideia era fazer a ordenação utilizando o método de ordenação mais eficiente, no teste foi utilizado o quicksort, que, em quase todos os casos, tem desempenho N log N. No fim de N iterações, a parte de ordenação teria um custo de N^2 Log N, porém é possível fazer melhor que isso.

A ideia da ordenação é feita, e logo em seguida é retirado o menor elemento, ou seja, apenas o menor elemento é de fato utilizado da ordenação feita. E se, ao invés de quicksort, fizermos um tipo de SelectionSort.

A cada iteração, encontra o menor elemento, e coloca na primeira posição, utiliza o elemento e a próxima primeira posição é incrementada.

```
void ordenarArray(Array* x, int lo){
    //encontrar o minimo e trocar com o primeiro
    float min;
    int pos;
    for(int i=lo;i<x->n;i++){
        if(retornaDistanciaS(x->array_no[i])<min || i==lo){
            min=retornaDistanciaS(x->array_no[i]);
            pos=i;
        }
    }
    trocaPosicaoArray(x, lo, pos);
}
```

Agora, a operação de ordenar têm desempenho linear (anteriormente N log N), e por lógica, quando feita em todos os vértices, têm desempenho $O(N^2)$, que inclusive é o que rege o desempenho de toda a operação.

2.4 Análise e Desempenho

Resultados de tempo para os casos de teste:

Dijkstra	Pequeno 1 (86 KB)	Pequeno 4 (8.593 KB)	Médio 1 (214 MB)	Médio 2 (214 MB)	Médio 3 (214 MB)	Médio 4 (309 MB)
Heap Binária	0.002175	0.177552	5.619822	5.228124	4.906362	9.218174
Array Sorted	0.002135	0.170662	4.629699	5.176462	4.831348	7.176714

A segunda implementação teve resultados melhores, mesmo, teoricamente sendo pior em custo. **$O((M+N)\log N)$ x $O(N^2)$**

Um grafo denso é um grafo que tem quase todas as arestas possíveis, e, a teoria diz que, um grafo com todas as arestas possíveis tem o quadrado do número de vértices.

Colocando então essa informação no custo do algoritmo temos:

$$M=N^2 \;\; \rightarrow \;\; O((M+N)\log N) = O(N^2+N)\log N) = \mathbf{O(N^2 \log N)}$$

Então, em um grafo denso, que é o caso dos exemplos de teste, o custo de execução da heap acaba sendo maior que o Array com o Selection Sort.

Além dessa parte teórica, o array simples tem um uso muito mais eficiente da cache, no caso da heap binária, com valores muito grandes, pequenos avanços podem significar grandes saltos, que resultam em mudança de página e acesso a disco.

3_ Proposta de Melhoria

Por fim, foi pedido para que fizesse uma melhoria no algoritmo original, o qual apresentava gargalos claros de desempenho, durante essa etapa foram propostas algumas possíveis melhorias, que foram executadas e analisadas para ver se de fato, melhoraram.

3.1 Tentativa 1: Sem ordenação desnecessária

Suposição:

Em um grafo denso, utilizando a segunda implementação, se a próxima menor distância para S não for alterada durante o processo de análise, então não é necessário fazer a ordenação, é só retirar o próximo elemento e seguir o algoritmo.

Resultados:

De fato, se colocarmos uma flag para detectar se a próxima menor distância foi alterada, e só fazer a ordenação se a flag for ativada, o número de ordenações cai consideravelmente. Por exemplo, no caso muito pequeno 2, foram feitas apenas 4 ordenações, das teóricas 8 necessárias.

Problema:

O problema dessa proposta é:

Para que o algoritmo simplesmente pegue o próximo elemento, quando não precisar ordenar, o array precisava estar pré-ordenado, então a proposta do SelectionSort não funcionaria, já que ele não ordena, somente encontra o menor elemento.

O algoritmo funciona se o método de ordenação utilizado for o QuickSort, porém, o corte da necessidade de ordenar pela proposta não consegue compensar o aumento do custo natural do algoritmo.

```
CASO PEQUENO 3 -> 0.172317 -> 0.166374 => 3,45%
CASO MÉDIO 3   -> 7.243023 -> 7.772594 => -7,31%
```

Suposição Negada.

3.2 Tentativa 2: Conexões em Array contínuo

Suposição:

Se trocarmos a lista de conexões de um nó, de uma lista encadeada, para um array contínuo, teremos menos gasto de memória, já que uma célula custa mais, além disso, menos acessos a cache, e, por consequência, maior velocidade.

Resultados:

O problema da mudança, é que independente da quantidade de conexões do vértice, a sua lista de conexões vai ser alocada com o tamanho máximo de N, em grafos esparsos é um desperdício de memória muito grande, e até em grafos densos a diferença pode ser sentida.

```
CASO PEQUENO 3 (MEMORY) -> 22.9MB / 24.1MB
```

Além da questão da memória, a velocidade também sofreu alterações para pior, e nesse ponto eu não tenho certeza do motivo, já que realmente os acessos a cache são diminuídos. Eu imagino que, por serem vetores de conexões muito grandes automaticamente os acessos a cache vão continuar equiparados.

O teste inclusive mostra uma piora significativa de desempenho:

```
CASO MÉDIO 2 -> 4.468201 -> 6.545307
CASO MÉDIO 3 -> 4.313614 -> 5.329737
```

Suposição Negada.

3.3 Melhoria 1: Finalizados não são reavaliados

Suposição:

Se utilizarmos um indicativo de que o vértice já foi visto e finalizado, então possíveis em conexões posteriores esse vértice não será mais analisado, reduzindo o número de operações totais

```
struct no{
    …
    bool finalizado;
};
```

```
while(tamanhoHeap(heap)>0){
        No* v_atual=removeHeap(heap);
        finalizarNo(v_atual);
        …
        while(1){
            …
            if(!estadoNo(aux)){
                    relaxaNo();
            }
            …
        }
    }
```

Resultados:

Ao custo de uma nova comparação, o número de chamadas da função de relaxamento é consideravelmente reduzido.

Se considerarmos um cenário de um grafo completamente denso, na primeira implementação, a função de relaxamento era chamada ~ N^2 vezes, uma para cada conexão de cada um dos N vértices.

Com a melhoria, a função de relaxamento agora é chamada metade das vezes que na original ~ $N^2/2$, já que a cada iteração, o número de vértices finalizados vai sendo incrementado, até chegar ao ponto de que todos foram finalizados.

```
CASO PEQUENO 3 -> 0.160085 / 0.174813 = 09,2%
CASO MÉDIO 4   -> 7.436667 / 9.218174 = 23,9%
```

Suposição Aceita!

Mas ainda é possível melhorar…

3.4 Melhoria 2: Modo de Leitura de Dados

Suposição:

Como visto no início do relatório, a leitura de dados é um problema porque é apresentada como uma lista de adjacência, logo a leitura é quadrática.

Se ao invés da leitura ser feita dado a dado, fazermos a leitura da linha inteira e então tratá-la internamente, teremos uma leitura linear, com operações internas quadráticas, sendo mais eficiente que puramente leitura quadrática.

```
char buffer[65536]; //buffer de 64K para ler linha por linha
if(fgets(buffer, sizeof(buffer), arq) == NULL) exit(1);
        for(int j = 1; j <= cont; j++){
            if(i!=j){
                while(//não número ou bomba) ptr++;
                …
                if(strncmp(ptr, "bomba", 5) == 0){
                    …
                }
                else {
                    peso = strtof(ptr, &ptr);
                }
                if(peso > 0 && i != j){
                    adicionarConexao(no_atual, retornaNoHeap(heap, j), peso);
                }
            }
        }
```

Resultados:

Com o uso de “fscanf” todas as conexões são lidas da entrada, logo, mais vezes se utiliza o I/O, mais precisamente N² vezes, além disso, o *parsing* é interno da função, custando caro por si só.

Já com “fgets” a linha é inteiramente lida para o buffer, reduzindo o número de acessos ao I/O para N, porém, o *parsing* por sua vez, se mantém na ordem de N². Mesmo assim, por não acessar o arquivo toda vez, apresenta uma eficiência muito maior.

Além disso, por ser lido em blocos de dados e manipulado por um ponteiro que é incrementado, a localidade de cache também é muito melhor aproveitada

CASO MÉDIO 1 -> 6.655925 / 3.676644 = 44,76%

3.5 Resultados em relação a melhoria

Resultados de tempo para os casos de teste:

Dijkstra	Pequeno 1 (86 KB)	Pequeno 4 (8.593 KB)	Médio 1 (214 MB)	Médio 2 (214 MB)	Médio 3 (214 MB)	Médio 4 (309 MB)
Melhoria	0.002983	0.176068	3.072097	3.075639	3.488150	5.332811

4_ Considerações Finais

Resultados de tempo para os casos de teste:

Dijkstra	Pequeno 1 (86 KB)	Pequeno 4 (8.593 KB)	Médio 1 (214 MB)	Médio 2 (214 MB)	Médio 3 (214 MB)	Médio 4 (309 MB)
Heap Binária	0.002175	0.177552	6.655925	5.228124	4.906362	7.115828
Array Sorted	0.002135	0.170662	5.112635	5.176462	4.831348	6.838628
Melhoria	0.002983	0.176068	3.072097	3.075639	3.488150	5.332811

O ganho da Melhoria em relação às duas outras implementações foi visto em todos os casos de teste, algumas constatações:

- No Médio 1 obteve **53,84%** de ganho para a primeira implementação;
- Em média, a **melhoria** em relação a **heap binária** foi de **37,24%**;
- E em relação ao **array** foi de **32,58%.**

O trabalho como um todo foi extremamente interessante de ser feito, e o aprendizado durante o processo foi genuíno.

Pensar em uma segunda implementação, e principalmente, propor uma melhoria, foram trabalhos com uma dificuldade considerável, tanto que em ambos os casos, várias possibilidades foram levantadas, testadas, e se não suficiente, descartadas.

Além do surpreendente uso do fgets ser tão melhor que o fscanf, algo que eu não tinha ciência.

OBS: Eu gostaria de ter analisado o custo em memória dos casos, porém, eu tive problemas de performance com a minha máquina e não fui capaz de rodar as maiores entradas com o valgrind ativo, em várias situações *crashando* o computador.

OBS 2: Também por conta do problema de desempenho da máquina, algumas das análises temporais podem ter sofrido alterações, em alguns casos eu encontrava tempos de execução muito diferentes em execuções sequenciais do mesmo caso de teste.

Para que esse relatório seja o mais correto possível, foram utilizados a mediana entre as 5 primeiras execuções.

Agradecimentos ao professor Luiz Antônio de Souza Junior.